Service Charter Document

Warehouse Management System

1. Purpose

To build a digital system that helps warehouse teams manage their daily work.

The system will:

- Track all items coming in and going out of the warehouse
- Show current stock levels at all times
- Keep records of all shippers and shipments
- Replace manual paper work with simple digital records
- Reduce mistakes in stock counting and shipping

2. Objective

- Make warehouse work faster by removing manual data entry
- Give warehouse staff one place to manage all items and shipments
- Show stock updates instantly whenever items come in or go out
- Prevent shipping mistakes by checking stock availability before dispatch
- Keep complete history of all shipments for future reference

2.1. System Scope

In Scope

- Warehouse item inventory management
- Incoming shipment recording
- Outgoing shipment processing
- Shipper profile management
- Stock monitoring and alerts
- Shipment history reporting
- API based backend system

Out of Scope (Phase 1)

- Barcode scanning integration
- Mobile application interface

- Automated warehouse robotics integration
- Multi-warehouse distributed inventory system
- AI demand prediction

3. Demand / Opportunity

Small and medium warehouses today still use paper logs or Excel sheets to manage their stock. **This causes problems like:**

- Wrong stock counts due to manual errors
- Delays in finding shipment history
- No way to know current stock without physical counting
- Difficulty coordinating with multiple shippers

Our Warehouse Management System solves these problems with a simple, reliable API that any warehouse can use. Since we are starting with just the backend API, warehouses can connect it to their own existing systems easily.

4. Business Requirement

4.1 User Authentication

- Users must login with email and password to access the system
- Each user gets a token that expires after some time for security
- Different access levels for warehouse staff vs managers

4.2 Item Management

- Add new items to warehouse with name, SKU code, category, and initial quantity
- View all items or search/filter by category or SKU
- Update item details like quantity, storage location, or status
- Delete items that are no longer in warehouse (managers only)

4.3 Shipping In (Goods Receiving)

- Record when new stock arrives at warehouse
- Capture shipper details, items received, quantities, and date/time
- Automatically increase stock levels when goods are received

- Show complete incoming shipment history

4.4 Shipping Out (Goods Dispatch)

- Record when stock leaves the warehouse
- Capture destination, shipper details, items sent, and quantities
- Check if enough stock exists before allowing shipment
- Automatically reduce stock levels when goods are dispatched
- Prevent shipping if stock is insufficient

4.5 Shipper Management

- Add new shippers with name, contact person, phone, and license number
- View, update, or deactivate shipper profiles
- Track all shipments done by each shipper

4.6 Stock Visibility

- Show current stock levels for all items in real-time
- Provide summary of total items, low stock alerts, and empty items

4.7 Reports

- View shipping history filtered by date range, shipper, or item
- Download shipment logs for record keeping

4.8 Functional Use Cases

Use Case ID	Use Case Name	Actor	Description
UC-01	User Login	Warehouse Staff	User logs in using email and password to access system
UC-02	Register User	Manager	Manager creates new system users
UC-03	Add Item	Warehouse Staff	Add new product/item into warehouse inventory

UC-04	Update Item	Warehouse Staff	Update quantity, category or location of item
UC-05	Delete Item	Manager	Remove item from system
UC-06	View Inventory	Staff	View all items with current stock
UC-07	Record Incoming Shipment	Staff	Add shipment when goods arrive
UC-08	Record Outgoing Shipment	Staff	Dispatch items to destination
UC-09	Manage Shippers	Manager	Create or update shipper profiles
UC-10	View Shipment History	Staff	View incoming/outgoing shipment records
UC-11	Generate Reports	Manager	Filter and export shipment data
UC-12	View Stock Summary	Staff	View real-time stock levels

5. Technical Requirement

5.1 User Authentication

- Create API routes for registration and login
- Use JWT tokens for secure session handling
- Store passwords in encrypted format
- Add middleware to check token on every protected request

5.2 Item Management

- Build API endpoints to create, read, update, and delete items
- Add validation to prevent duplicate SKU codes
- Allow filtering and searching in list endpoints

5.3 Shipping In

- Build API endpoint to record incoming shipments
- Update item quantities automatically in same transaction
- Store shipper ID, items list, quantities, and timestamp

5.4 Shipping Out

- Build API endpoint to record outgoing shipments
- Check stock availability before processing
- Deduct quantities and save shipment record in same transaction
- Return error with details if stock insufficient

5.5 Shipper Management

- Build CRUD endpoints for shipper profiles
- Ensure shipper ID is valid before using in shipments

5.6 Stock and Reports

- Create endpoint to get real-time stock levels
- Build reporting endpoint with date filters and search options
- Ensure data consistency using database transactions

5.7 Non-Functional Requirements

Performance

- API response time should be < **2 seconds** for normal requests.
- The system should support **100+ concurrent API requests**.

Security

- Passwords encrypted using **BCrypt hashing**.
- JWT tokens expire after **30 minutes**.
- Role Based Access Control for sensitive operations.

Availability

- System uptime target: **99% availability**.

Scalability

- Backend designed using **microservice-ready architecture**.
- Docker containers allow easy scaling.

Reliability

- All stock updates use **database transactions** to avoid inconsistencies.

Usability

- APIs documented using **Swagger UI**.

6. Technological Requirement

6.1 Backend

- Java 17
- Spring Boot framework
- Spring Security for authentication
- Spring Data JPA for database operations

6.2 Database

- MySQL for reliable data storage
- ACID compliance to ensure no data loss during transactions

6.3 Authentication & Security

The system uses **JWT (JSON Web Tokens)** for secure authentication and authorization of users accessing the API.

Authentication Process

1. A user logs in using **email and password**.
2. The backend verifies the credentials against the stored user data in the database.
3. If authentication is successful, the system generates a **JWT token**.
4. The token is returned to the client and must be included in the **Authorization header** of every protected API request.
5. The server validates the token before allowing access to secured endpoints.

Security Mechanisms

Password Encryption

- User passwords are encrypted using **BCrypt hashing algorithm** before storing in the database.
- Plain text passwords are never stored.

JWT Token Validation

- Every request to protected APIs passes through a **JWT authentication filter**.

- The filter verifies:
    - Token validity
    - Token expiration
    - User identity

Token Expiry

- JWT tokens are configured with an **expiration time (e.g., 30–60 minutes)** to reduce security risks.
- Expired tokens require the user to log in again.

Role-Based Access Control (RBAC)

- Different user roles are defined:
    - **Manager**
    - **Warehouse Staff**

- Certain operations such as **deleting items or managing users** are restricted to managers only.

Spring Security Integration

- The authentication system is implemented using **Spring Security**.
- Spring Security manages:
    - login authentication
    - request filtering
    - role-based authorization

Secure API Communication

- All API requests are expected to be sent using **HTTPS** to protect data during transmission.

6.4 Tools

- Visual Studio Code for coding
- Git & GitHub for version control
- Postman for API testing
- Swagger for API documentation

6.5 Deployment

- Docker for containerization
- AWS Cloud (EC2, RDS) for hosting

6.6 System Architecture

The system follows a **3-Layer Architecture**:

1. Presentation Layer

- REST API endpoints
- Swagger API documentation

2. Application Layer

- Spring Boot services
- Business logic
- Authentication & authorization

3. Data Layer

- MySQL database
- Spring Data JPA repositories

Architecture Type:
RESTful API Based Backend System

6.7 Core Database Entities

Entity	Description
User	System users with login credentials
Item	Warehouse inventory items
Shipper	Logistics partners transporting goods

Shipment_In	Records incoming shipments
Shipment_Out	Records outgoing shipments
Shipment_Items	Items associated with shipments

7. Stakeholders

Stakeholder	Name	Count
Warehouse Owner/Manager	To be assigned after client finalization	1
Warehouse Staff	End users from client side	3-5
Shippers/Logistics Partners	External users	Multiple
Developers	Om Panchal, Anushka Jadhav, Uday Sapate, Kiran Nandi	4
QA / Tester	Yash Bisen	1
Course Instructor	Jayesh Raut	1
Reviewer	Srivaths Iyer	1
Cloud Service Provider	AWS	1

8. Resources Needed

8.1 Documentation

- Java Spring Boot documentation
- MySQL documentation
- JWT authentication guides
- AWS setup guides

8.2 Human Resource

Role	Count	Names
Backend Developers	3	Kiran Nandi, Anushka Jadhav, Om Panchal
Frontend Developer	1	Uday Sapate
QA / Testing	1	Yash Bisen
External Testers	2	Varad Kolhe, Ishwari Mahajan
Database Designer	2	Kiran Nandi, Om Panchal
Documentation	2	Uday Sapate, Anushka Jadhav
Deployment	2	Kiran Nandi, Om Panchal
Project Management	1	Om Panchal

9. PESTEL Analysis

9.1 Political

No direct political impact on our project. The system follows standard data storage practices that comply with local IT regulations.

9.2 Economic

- **For users:** Warehouses can save money by reducing manual labor hours spent on stock counting and paper work. Less stock mismatch means less money lost in errors.
- **For us:** Development cost is mainly team effort and cloud hosting. Once built, maintenance cost is low.

9.3 Social

Warehouse staff can focus on actual work instead of maintaining paper records. Junior staff can learn and adapt quickly since the system is simple to use via APIs. Reduces frustration caused by stock mismatch blame games.

9.4 Technological

Built on Java Spring Boot which is stable and widely used. MySQL ensures data reliability. The system is API-first so it can connect to any frontend or mobile app in future. If any technology updates, we can upgrade gradually.

9.5 Legal

- Data stored belongs to the warehouse client, not us
- We follow standard data protection practices
- If commercialized later, we will ensure compliance with data protection laws

9.6 Environmental

No direct environmental impact. Digital records reduce paper usage. Cloud hosting uses shared resources efficiently.

10. Risk Analysis

Risk	Description	Mitigation Strategy
Data Loss	Accidental deletion of items or shipments	Regular automated database backups, soft delete option instead of permanent delete
API Downtime	System unavailable due to server or database issues	Host on AWS with auto-recovery, keep backup plan ready
Stock Mismatch	Wrong stock levels due to failed transactions	Use database transactions to ensure all steps complete or none complete
Security Breach	Unauthorized access to warehouse data	Strong password encryption, JWT with short expiry, role-based access control
Budget Overrun	Higher than expected AWS costs	Monitor usage daily, set budget alerts, use free tier as much as possible
Scope Creep	Client asking for features beyond original plan	Clear sign-off on requirements, document change requests separately
Performance Issues	Slow response when many requests come together	Add database indexing, optimize queries, plan for load balancing if needed
Integration Problems	Client unable to connect API to their system	Provide clear API documentation with examples, offer sample code
Team Member Unavailable	Illness or emergency during critical phase	Cross-train team members, maintain code documentation, regular backups of work

Data Loss During Shipping	Stock update fails but shipment record saves	Always use transactions, add validation before processing, log all failures

11. Timeline / Milestone

Phase	Milestone	Tasks	Timeline
1	**Planning & Setup**	• Understand all project requirements clearly • Design database schema • Set up GitHub repository • Configure development environment • Create project structure in Spring Boot	**Week 1** (March 1 – March 7)
2	**Core Development – Item & Shipper**	• Build Item CRUD APIs • Build Shipper CRUD APIs • Add validation and error handling • Test all endpoints manually • Document APIs using Swagger	**Week 2** (March 8 – March 14)
3	**Shipping Logic & Reporting**	• Build Shipping In API with stock update • Build Shipping Out API with stock validation • Add stock summary endpoint • Develop report endpoints with filters • Test all transaction scenarios	**Week 3** (March 15 – March 21)
4	**Testing, Debugging & Deployment**	• Complete API integration testing • Fix bugs and optimize database queries • Set up AWS EC2 and RDS • Deploy application using Docker • Prepare final documentation	**Week 4** (March 22 – March 31)

12. RACI Chart

		Roles						
	Activity / Members	Om Panchal (PM / Backend / DB / Deploy)	Anushka Jadhav (Backend / Docs)	Uday Sapate (Frontend / Docs)	Kiran Nandi (Backend / DB / Deploy)	Yash Bisen (QA)	Jayesh Raut	Srivaths Iyer
Vision	Requirement Analysis & Planning	A	C	C	C	I	I	C
	Design Database	A	C	I	R	I	C	I
	Design System Architecture	A	R	I	R	I	C	C
Administration	User Authentication (JWT Implementation)	A	R	I	R	C	I	C
	Item Management API (CRUD)	A	R	I	R	C	I	I
	Shipper Management API	A	R	I	R	C	I	I
	Shipping In API (Stock Increase Logic)	A	R	I	R	C	I	C
	Shipping Out API (Stock Validation Logic)	A	R	I	R	C	I	C
	Stock Summary & Visibility Endpoint	A	R	I	R	C	I	C
Enhancement Proces	Reports & Filter APIs	A	R	I	R	C	I	C
	API Validation & Error Handling	A	R	I	R	C	I	I
	API Testing (Postman & Integration)	C	C	I	C	A	I	C
	Swagger Documentation	C	R	R	I	I	C	C
	GitHub Version Control Management	A	C	C	R	I	I	I
	Docker Configuration	A	I	I	R	C	I	I
	AWS EC2 & RDS Setup	A	I	I	R	C	I	C

R = Responsible (does the work)

A = Accountable (answers for completion)

C = Consulted (gives input before decision)

I = Informed (told after decision)

13. Budget

Project Duration: 1 Month (March 2025)

Service	Purpose	Monthly Cost (INR)

Amazon EC2 (t3.micro)	Host Spring Boot application and run backend APIs	₹700
Amazon RDS MySQL (db.t3.micro)	Managed relational database service for storing user data, bookings, and application records	₹1500
Amazon S3	Store backups, logs, uploaded files, and other static resources	₹300
Amazon EBS (30 GB)	Persistent storage attached to the EC2 instance for application data and OS	₹250
Data Transfer	Internet data transfer for API calls, client requests, and responses	₹300
Domain Name (.com)	Custom domain for accessing the API or application	₹75
Total Estimated Cost		**₹3125 / month (approx.)**

Basis of Calculation:

1. Amazon EC2 (t3.micro)

- Price ≈ **$0.0104 per hour**
- Monthly hours ≈ **730**

Calculation:
0.0104 × 730 = $7.59

Convert to INR (₹83/$):
7.59 × 83 ≈ ₹630 ≈ ₹700

2. Amazon RDS MySQL (db.t3.micro)

- Price ≈ **$0.017 per hour**

- Monthly hours ≈ **730**

Calculation:
0.017 × 730 = $12.41

Convert to INR:
12.41 × 83 ≈ ₹1030

Add storage (20GB ≈ ₹400)

Total:
₹1030 + ₹400 ≈ ₹1500

3. Amazon S3

Storage cost:

- 50 GB storage
- Price ≈ **$0.023 per GB**
  Calculation:
  50 × 0.023 = $1.15

Convert to INR:
1.15 × 83 ≈ ₹95

Add request & backup cost ≈ **₹200**

Total:
₹300

4. Amazon EBS (30 GB)

Price ≈ **$0.08 per GB**

Calculation:
30 × 0.08 = $2.4

Convert to INR:
2.4 × 83 ≈ ₹199 ≈ ₹250

5. Data Transfer

Assuming **50 GB outgoing data**

Price ≈ **$0.09 per GB**

Calculation:
50 × 0.09 = $4.5

Convert to INR:
4.5 × 83 ≈ ₹373 ≈ ₹300–₹400

6. Domain Name (.com)

Typical cost:

₹800 – ₹900 per year

Monthly:
900 ÷ 12 ≈ ₹75

Note: If a project is deployed only for testing/demo, we can stop instances when not in use to reduce cost. The development team is part of an academic project, so no salary cost included.

In-House Server Cost Estimation

Component	Purpose	Estimated Cost (INR)
Server Hardware (Mid-range Server PC)	Run backend application and database	₹60,000
Storage (1 TB SSD/HDD)	Store database, logs, and system files	₹6,000
UPS Power Backup	Prevent shutdown during power failure	₹5,000
Network Equipment (Router / Switch)	Internal system connectivity	₹3,000

Electricity Cost	Power consumption for server operation	₹700 / month
Maintenance & IT Support	Hardware servicing and updates	₹500 / month

Short Cost Calculation

1. Server Hardware

A mid-range server with:

- **Intel i5 / Ryzen 5**
- **16 GB RAM**
- **1 TB storage**

Estimated cost:
₹55,000 – ₹65,000

Used in table: **₹60,000**

2. Storage

Additional **1 TB SSD/HDD** for database and backups.

Approximate cost:
₹5,000 – ₹7,000

Used in table: **₹6,000**

3. UPS Power Backup

UPS ensures system stability during power outages.

Cost:
₹4,000 – ₹6,000

Used in table: **₹5,000**

4. Network Equipment

Router or switch required for warehouse internal network.

Cost:
₹2,000 – ₹4,000

Used in table: **₹3,000**

5. Electricity Cost

Typical server power consumption: **150–200 W**

Monthly energy usage:

200 W × 24 hours × 30 days
= **144 kWh**

Electricity rate ≈ **₹5 per unit**

Calculation:
144 × 5 ≈ **₹720/month**

Used in table: **≈ ₹700/month**

6. Maintenance Cost

Includes:

- Hardware servicing
- Software updates
- IT support

Estimated: **₹500/month**

Total Cost

Initial Setup Cost

Server + Storage + UPS + Network

₹60,000 + ₹6,000 + ₹5,000 + ₹3,000
= ₹74,000

Monthly Operational Cost

Electricity + Maintenance

₹700 + ₹500
= ₹1,200 / month